SGEIA

Smart Grid Energy Intelligence Assistant

Project Story & Technical Documentation

Tejeswari S K

Prodapt AFDE Capstone | May 2026

tejeswari.sk@prodapt.com

Section 1: What is SGEIA?

SGEIA (Smart Grid Energy Intelligence Assistant) is an AI-powered web dashboard that lets power grid operators ask questions in plain English and get clear, data-backed answers in seconds. Instead of manually searching through logs and spreadsheets, an operator just types "Why is Zone A showing instability?" and the system finds the relevant data, analyses it using machine learning, and explains what is happening and why.

Section 2: The Problem It Solves

Power grid operators deal with three problems every day: information overload (thousands of sensor alerts), slow diagnosis (hours to find fault root causes), and disconnected tools (stability data, meter readings, and incident reports all in separate systems with no AI to connect them). SGEIA solves all three by combining intelligent search, machine learning, and a natural language chat interface into one platform.

Section 3: How Data Flows — The RAG Pipeline

This is the core of how SGEIA works. Every question goes through this pipeline:

USER INPUT ↓
CSV DATASETS (DS1: smart_grid_stability_augmented.csv + DS2: household_power_consumption.csv)
↓
CHUNKING (Each incident description = 1 chunk, avg 50 words)
↓
EMBEDDING (all-MiniLM-L6-v2, 384-dim vectors, runs locally on CPU)
↓
INDEXING (BM25 keyword index + ChromaDB vector index built simultaneously)
↓
VECTOR STORAGE (ChromaDB persistent store on disk, 200 incident vectors)
↓
[AT QUERY TIME] USER ASKS QUESTION
↓
HYBRID SEARCH (BM25 keyword + ChromaDB semantic → RRF fusion → top 8 results)
↓
LLM (Groq llama-3.1-8b-instant receives: question + retrieved chunks + DS1 stats + DS2 stats)
↓

OUTPUT (validated answer shown to user with source citations)

Step-by-Step Detail

Step 1 — User Input

The operator types a question in plain English, for example: "List the last 10 high voltage readings" or "Why is Zone A unstable?"

Step 2 — CSV Datasets

The system has two datasets as its knowledge base:

- **DS1 (smart_grid_stability_augmented.csv) — 60,000 rows of grid stability telemetry**
- **DS2 (household_power_consumption.csv) — 2 million rows of smart meter readings**

These CSV files are the source of truth. All answers are grounded in this real data.

Step 3 — Chunking

The 200 synthetic incident records (generated from DS1 and DS2) are broken into chunks. Each incident description becomes one chunk — approximately 40 to 80 words. No further splitting is needed because the incident descriptions are already an ideal size for embedding.

Step 4 — Embedding

Each chunk is converted into a 384-dimensional vector (a list of numbers) using the all-MiniLM-L6-v2 sentence embedding model. This model runs entirely on the local CPU with no internet connection needed — important because the corporate proxy blocks external embedding APIs. The vector captures the meaning of each incident, not just the words.

Step 5 — Indexing

Two indexes are built simultaneously from the same 200 incidents:

- BM25 keyword index — stores the words for fast exact matching
- ChromaDB vector index — stores the 384-dim vectors for semantic similarity search

Step 6 — Vector Storage

ChromaDB saves all 200 incident vectors to disk permanently. They survive server restarts and never need to be re-embedded unless the incident data changes.

Step 7 — Hybrid Search (at query time)

When a user asks a question, two searches happen in parallel:

- BM25 finds exact keyword matches (Zone_A, INC-042, transformer, full_outage)

- ChromaDB finds semantically similar incidents (even if the words differ)

Reciprocal Rank Fusion (RRF) combines both result lists, removes duplicates, and selects the top 8 most relevant incidents.

Step 8 — LLM Processing

The Groq llama-3.1-8b-instant model receives all of this combined:

- The user's question
- The 8 retrieved incident records (with IDs, zones, descriptions)
- DS1 stability statistics (health score, XGBoost prediction, SHAP feature importance)
- DS2 smart meter statistics (voltage range, anomaly count, consumption data)

The LLM reads all of this and writes a clear, specific answer that cites actual numbers and incident IDs.

Step 9 — Output

Before showing the answer, DeepEval checks faithfulness (does the answer use real data?), an LLM-as-Judge scores it 1-5 (must be 3+), and output guardrails scan for harmful content or PII. The validated answer is then shown to the user and stored in cache for instant reuse if the same question is asked again.

Section 4: Why Two Datasets?

A power grid has two sides. DS1 covers the SUPPLY SIDE — how generators, transmission lines, and grid nodes behave under different load conditions. DS2 covers the DEMAND SIDE — what electricity consumers (homes and buildings) actually use minute by minute.

Using only DS1 would mean we understand grid physics but not consumer demand. Using only DS2 would mean we see consumption patterns but cannot explain grid instability. Using BOTH gives a complete picture — for example, we can identify that Zone A is simultaneously experiencing generator instability (DS1) and unusually high household demand (DS2), which together explain the voltage deviation incident.

Section 5: Why New Columns Were Added

Both original datasets lacked the operational context needed for real grid management.

DS1 Original vs Added

Original columns: tau1-4, p1-4, g1-4, stab, stabf (only physics features, no location or time)

New columns added: timestamp, region (Zone_A/B/C/D), equipment_type, transformer_status, grid_frequency, outage_event

Why: Real grid operations require knowing WHEN a reading occurred, WHERE in the grid, WHAT equipment was involved, and WHAT type of event occurred. Without these, the data cannot be filtered by zone or equipment type in the RAG pipeline.

DS2 Original vs Added

Original columns: date, time, power_consumption, reactive_power, voltage, current, sub_metering_kitchen, sub_metering_laundry, sub_metering_hvac (no grid context)

New columns added: region, equipment_type, transformer_status, outage_event, demand_load, grid_frequency

Why: The original dataset had no connection to the grid at all. Adding zone and event columns allows smart meter anomalies to be linked to grid stability events, enabling cross-dataset analysis.

Section 6: Techniques Used and Why

XGBoost for Grid Stability Classification

XGBoost classifies each DS1 row as stable or unstable and gives a probability score. It was chosen over Logistic Regression (too simple for non-linear tau/p/g relationships), Neural Networks (require far more data, cannot explain predictions), and Random Forest (slower, less effective on the 63.8%/36.2% imbalanced dataset). XGBoost produces SHAP values that show exactly which feature — for example high tau1 meaning slow reaction time — drove each instability prediction.

Isolation Forest for Anomaly Detection

Isolation Forest detects unusual patterns in DS2 smart meter data without needing pre-labelled anomalies. DS2 has 2 million rows with no labels indicating which readings are abnormal. Isolation Forest is unsupervised — it finds outliers by measuring how easy it is to isolate a data point from the rest. It scales efficiently to millions of rows and gives a numeric anomaly score per reading.

Hybrid RAG — BM25 plus ChromaDB

BM25 keyword search catches exact matches like zone names, incident IDs, and equipment types. ChromaDB semantic search catches conceptually similar incidents even when different words are used. Combining both with RRF fusion gives better recall than either method alone. The all-MiniLM-L6-v2 embedding model was chosen because it runs locally on CPU with no internet required.

LangGraph Multi-Agent Orchestration

Different questions need different agents. A question about incidents needs only the retrieval agent. A question about stability needs XGBoost. A question about consumption needs Isolation Forest. LangGraph routes each question to only the relevant agents, making the system faster and more accurate than calling all agents every time.

Section 7: Guardrails and Safety

A four-layer input guardrail pipeline validates every question before it enters the RAG pipeline:

- Layer 1 — Format Check: Is it long enough? Not empty? Under 1000 characters?
- Layer 2 — Harmful Content: Violence, threats, weapons, illegal content → BLOCKED immediately
- Layer 3 — Domain Check: Not about power grids or energy? → REJECTED with explanation
- Layer 4 — PII Masking: Phone numbers, emails, GPS coordinates, and ID numbers replaced with [PHONE], [EMAIL], [GPS_MASKED]

Output guardrails scan every LLM response before it reaches the user.

Section 8: DeepEval Quality Validation

LLM answers are evaluated for:

- Faithfulness — does the answer cite actual data from retrieved incidents?
- Quality — LLM-as-Judge scoring 1-5, threshold 3
- Safety — no harmful content, no system prompt leakage

Answers that fail are retried once before being shown to the user.

Section 9: Summary

SGEIA demonstrates that multi-agent AI with a proper RAG pipeline can transform reactive grid operations into proactive, explainable intelligence. The complete flow:

User Input → CSV Datasets → Chunking → Embedding → Indexing → Vector Storage → Hybrid Search → LLM → DeepEval → Output

...ensures every answer is grounded in real data, validated for quality, and safe for operational use.

Built at Prodapt AFDE Capstone | May 2026 | Tejeswari S K | tejeswari.sk@prodapt.com